

BÁO CÁO TÓM TẮT TIẾN ĐỘ THỰC HIỆN ĐỀ TÀI

- **Dự án:** Xây dựng nền tảng ODS thời gian thực phục vụ phân tích dữ liệu lớn
- **GitHub Repository:** [dducsw/vdt-project](#)
- **Thành viên thực hiện:** Lê Đình Đức - VDT
- **Mentor hướng dẫn:** Nguyễn Ngọc Long - VTS

2. XÂY DỰNG PIPELINE INGEST VÀ XỬ LÝ DỮ LIỆU

Sinh và thu thập dữ liệu

- **Dữ liệu nghiệp vụ (OLTP):** Dữ liệu giao dịch eCommerce mô phỏng từ tập dữ liệu gốc [TheLook eCommerce \(Kaggle Looker Ecommerce BigQuery Dataset\)](#), lưu trữ trên cơ sở dữ liệu nguồn PostgreSQL ([thelook_db](#)) bao gồm các bảng: [users](#), [products](#), [orders](#), [order_items](#), [distribution_centers](#), [inventory_items](#). Do đây là nguồn dữ liệu có cấu trúc và được kiểm soát chặt chẽ bởi các ràng buộc giao dịch từ hệ thống OLTP nên độ chính xác rất cao và ít xảy ra sai sót.
- **Dữ liệu hành vi (Clickstream):** Sử dụng script giả lập (Python Datagen) sử dụng thư viện [Faker](#) và [confluent-kafka](#) sinh liên tục các sự kiện tương tác người dùng thời gian thực (xem sản phẩm, thêm giỏ hàng, thanh toán, hủy đơn, v.v.).

Xây dựng pipeline ingest dữ liệu realtime

- **Luồng CDC (Change Data Capture):** Cấu hình [Debezium](#) theo dõi Write-Ahead Log (WAL) của PostgreSQL nguồn. Mọi thay đổi dữ liệu (Insert, Update, Delete) được chụp tức thì dưới dạng sự kiện JSON và gửi vào các topic tương ứng của Kafka làm hàng đợi đệm.
- **Luồng Clickstream:** Các sự kiện hành vi từ Datagen (đóng vai trò giả lập một Collector/BFF service nhận request HTTP từ client và ghi nhận) được bắn trực tiếp vào topic [clickstream-events](#) trên Kafka để giảm thiểu độ trễ mạng và tiết kiệm tài nguyên JVM. Trong thực tế sản xuất, client (trình duyệt, app di động) không kết nối trực tiếp đến cổng TCP của Kafka Broker vì lý do bảo mật và phân quyền, mà sẽ gửi qua giao thức HTTPS tới một tầng đệm API Gateway, BFF (Backend-For-Frontend) hoặc Confluent REST Proxy trước khi đẩy vào Kafka (*cái này nếu em có thời gian thì em sẽ cải thiện*).
- **Cơ chế nạp tối ưu:** Sử dụng [Doris Native Routine Load](#) để nạp trực tiếp luồng CDC từ Kafka vào Apache Doris mà không qua Spark, giúp tiết kiệm bộ nhớ JVM và tối ưu hóa CPU cho toàn hệ thống.

Thực hiện transform và xử lý dữ liệu phục vụ analytics

- **Xử lý dòng với PySpark Structured Streaming:**
 - Đọc luồng dữ liệu clickstream từ Kafka theo micro-batches 5 giây.
 - Phân tách dữ liệu lỗi định dạng đẩy sang bảng deadletter ([ods_clickstream_deadletter](#)).
 - Áp dụng cơ chế **Watermark (10 phút)** và loại bỏ trùng lặp sự kiện dựa trên khóa ([id](#), [event_timestamp](#)) để đảm bảo xử lý chính xác 1 lần (exactly-once).
 - Parse trường URI để bóc tách thông tin trang ([page_type](#)) và ID sản phẩm tương tác ([product_id](#)).
 - Ghi dữ liệu sạch xuống Doris dưới định dạng Stream Load.
- **Xử lý Delete CDC:** Đồng bộ hành động xóa từ nguồn PostgreSQL thông qua cột ẩn [__DORIS_DELETE_SIGN__](#) của Doris Unique Key tại tầng Routine Load.

- **Triết lý thiết kế:** Hệ thống không lạm dụng biến đổi dữ liệu sâu trên Spark mà chuyển sang cơ chế **Transform-on-Query** tận dụng sức mạnh tính toán song song MPP & Vectorized Engine của Doris. Điều này giúp giữ cho pipeline ghi nhận dữ liệu (ingestion pipeline) luôn có độ trễ cực thấp (sub-second) và dễ dàng mở rộng.

3. THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG ODS

Thiết kế schema dữ liệu và danh sách các bảng vật lý trong Apache Doris

Hệ thống lưu trữ dữ liệu tại database `thelook_dw` bao gồm 8 bảng vật lý:

- **Mô hình Duplicate Key (Append-only):** Tối ưu cho ghi dữ liệu hành vi lớn, không tốn chi phí đối chiếu khóa khi nạp.
 - `dwd_clickstream_events`: Lưu chi tiết sự kiện clickstream sạch đã bóc tách URI.
 - `ods_clickstream_deadletter`: Lưu các bản tin clickstream thô bị lỗi định dạng (Dead-letter Queue).
- **Mô hình Unique Key với cơ chế Merge-on-Write (WoW):** Áp dụng cho dữ liệu nghiệp vụ đồng bộ qua CDC, tự động cập nhật đề khi trùng khóa chính và tối ưu tốc độ đọc gấp 3-10 lần nhờ Delete Bitmap.
 - `dim_users`: Dữ liệu thông tin khách hàng (từ OLTP `users`).
 - `dim_products`: Dữ liệu danh mục sản phẩm (từ OLTP `products`).
 - `dim_distribution_centers`: Dữ liệu trung tâm phân phối (từ OLTP `distribution_centers`).
 - `fact_orders`: Thông tin tổng quan đơn đặt hàng (từ OLTP `orders`).
 - `fact_order_items`: Chi tiết từng mặt hàng trong đơn (từ OLTP `order_items`).
 - `fact_inventory_items`: Trạng thái và chi tiết sản phẩm trong kho (từ OLTP `inventory_items`).

Tối ưu partitioning, indexing và storage layout

- **Partitioning:** Thiết lập phân vùng Range theo ngày (`event_date`) kết hợp tính năng phân vùng động (Dynamic Partitioning) để tự động tạo mới phân vùng và thu hồi dữ liệu hết hạn.
- **Bucketing:** Phân cụm Hash theo khóa phân tán (ví dụ: `session_id`, `product_id`) chia nhỏ dữ liệu thành các Tablet vật lý độc lập.
- **Indexing:**
 - Xây dựng chỉ mục tiền tố (**Prefix Index**) phục vụ tìm kiếm nhị phân khối dữ liệu nhanh chóng.
 - Cấu hình **Inverted Index** (chỉ mục đảo) trên cột `event_type` phục vụ tìm kiếm từ khóa.
 - Tạo chỉ mục xác suất **Bloom Filter** trên các cột khóa lớn (`session_id`, `user_id`) để bỏ qua nhanh các block I/O không chứa dữ liệu.
- **Colocate Join:** Đồng cấu trúc phân cụm bám giữa các bảng lớn để thực thi phép JOIN nội bộ tại BE, loại bỏ chi phí truyền tải qua mạng (Network Data Shuffle).

Triển khai hệ thống query phục vụ dashboard và analytics realtime

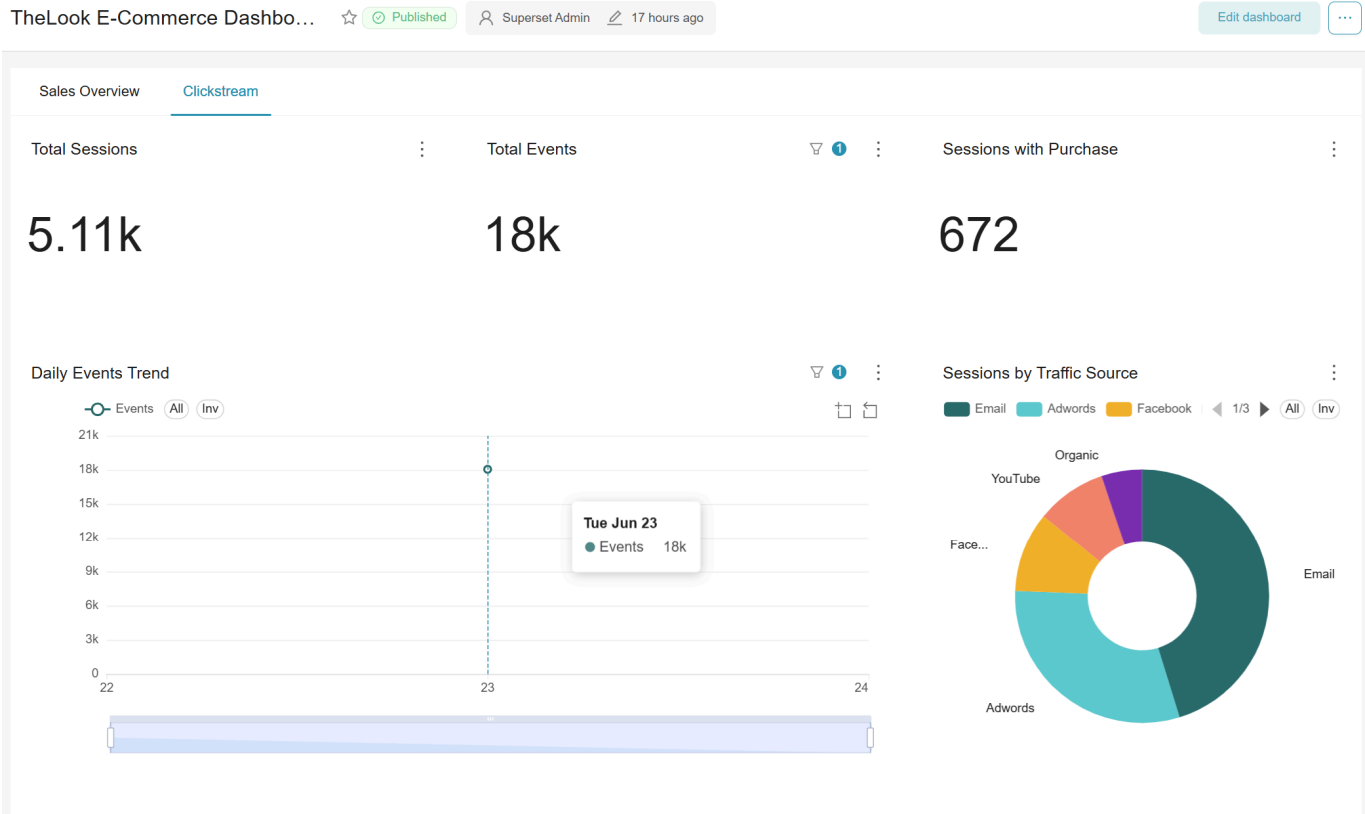
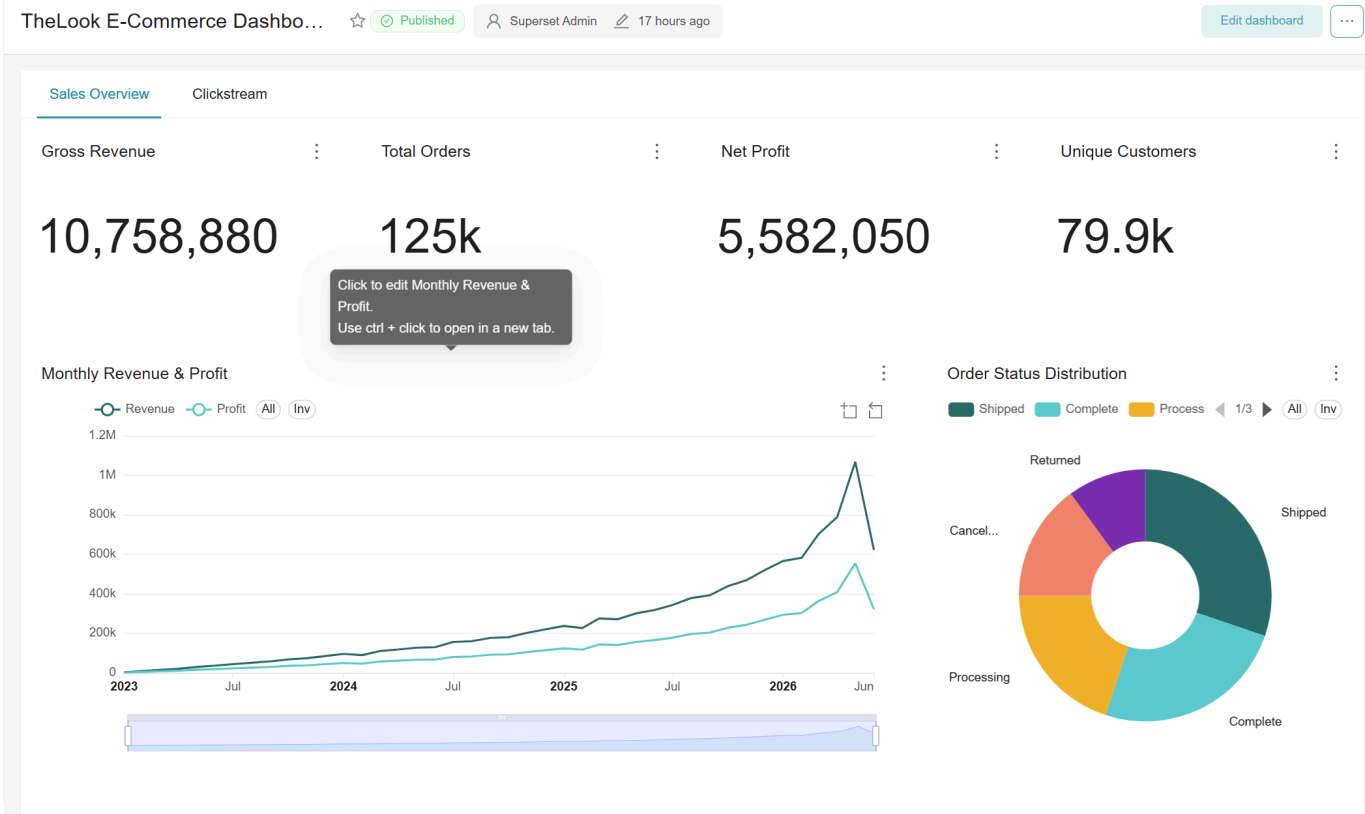
- Triển khai các **View phân tích (DWS Layer)** trong Doris để thực hiện tính toán tổng hợp thay cho pipeline:
 - `dws_clickstream_window_agg`: Tính toán dữ liệu clickstream theo cửa sổ trượt 5 phút.
 - `dws_clickstream_sessions`: Tổng hợp hành vi tương tác và xác định danh mục ưu thích theo phiên.

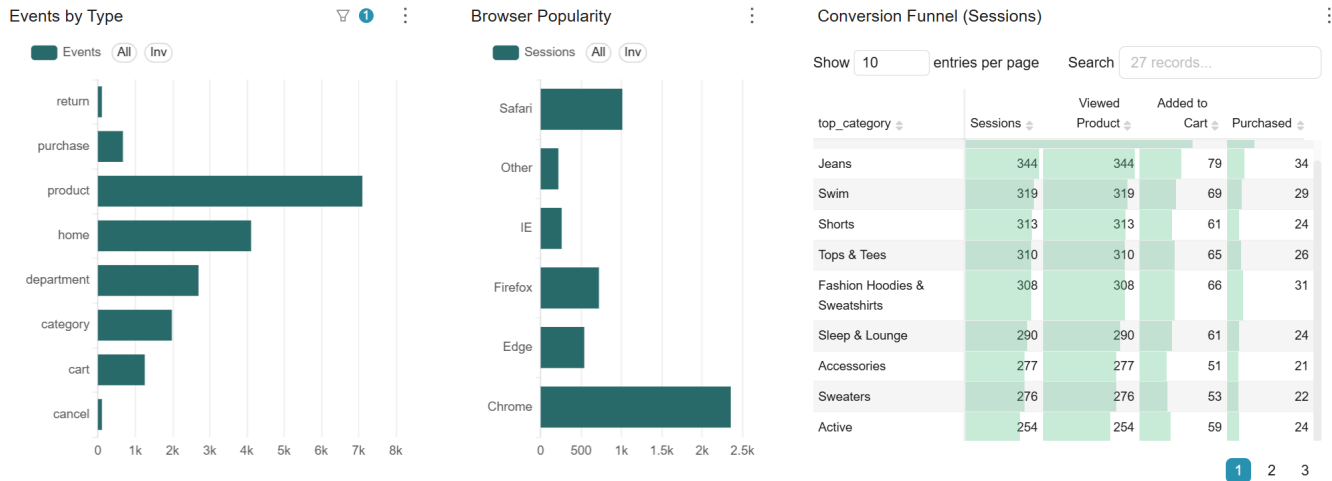
- `dws_sales_performance_flat`: Phép JOIN phẳng phục vụ phân tích doanh thu và lợi nhuận kết hợp dim/fact.
 - `dws_sales_overview_hourly`: Theo dõi hiệu năng bán hàng theo giờ.
-

4. QUERY, VISUALIZATION VÀ MONITORING

Xây dựng dashboard trực quan hóa dữ liệu (Đang cải thiện)

- Sử dụng **Apache Superset** kết nối qua cổng MySQL tương thích của Doris FE.
- Thiết lập dashboard trực quan hóa: Phễu chuyển đổi hành vi (Conversion Funnel), doanh số bán hàng theo giờ, hiệu suất tồn kho và top sản phẩm xu hướng thời gian thực.



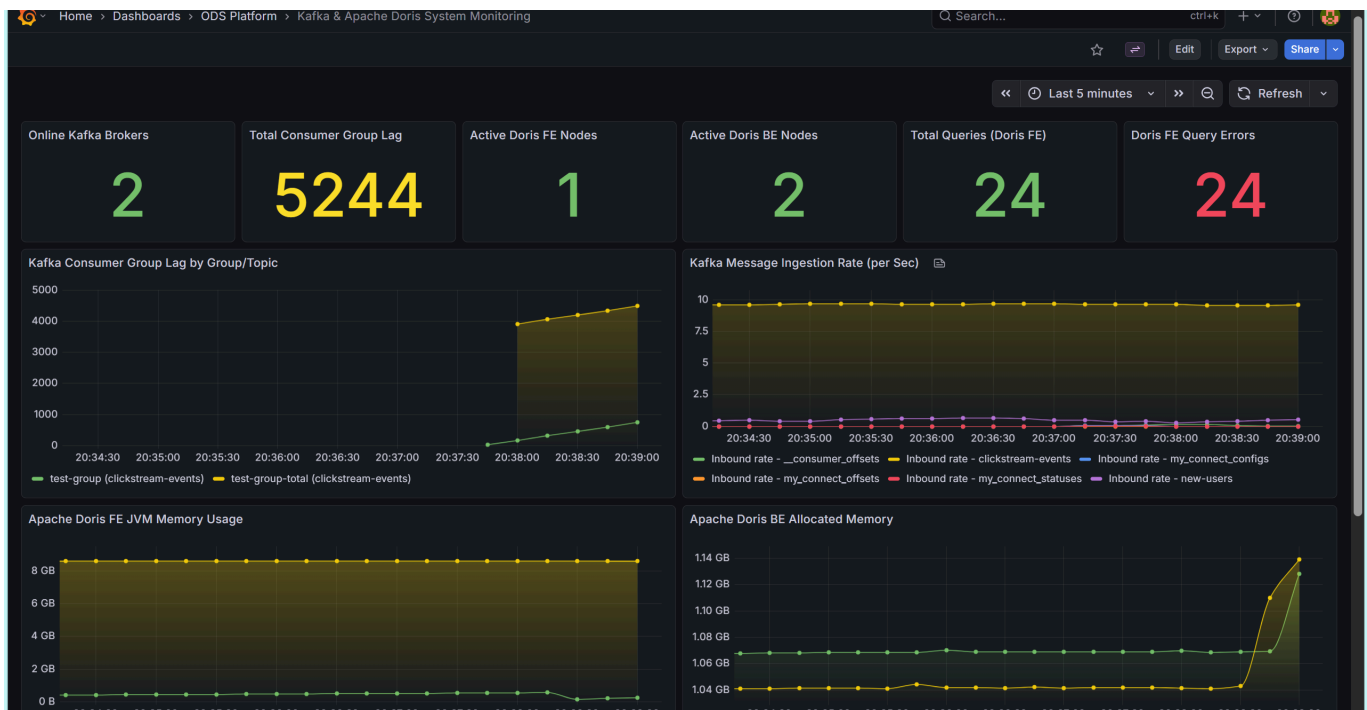


Thực hiện các truy vấn analytics realtime (Đang cải thiện)

- Chạy trực tiếp các câu lệnh phân tích SQL phức tạp trên Doris:
 - Phân tích tỷ lệ chuyển đổi từng bước trong chuỗi mua hàng.
 - Đo đếm độ trễ và thông lượng nạp dữ liệu tức thời trong 5 phút.
 - Thống kê top 10 sản phẩm thịnh hành trong 15 phút qua.

Theo dõi và giám sát hiệu năng hệ thống (Đang cải thiện)

- Giám sát Kafka Consumer Lag của các topic để đảm bảo không bị nghẽn dữ liệu.
- Theo dõi tải CPU/RAM của Doris BE và FE trong suốt quá trình ghi nhận.



Giải thích các chỉ số trên Grafana Dashboard (từ kết quả đo thực tế trong hình):

- Online Kafka Brokers (2):** Cho biết cả 2 Kafka Broker trong cụm đang hoạt động bình thường, đảm bảo tính sẵn sàng cao và phân vùng chịu lỗi.
- Total Consumer Group Lag (5.244):** Tổng số bản tin sự kiện đang bị tồn đọng trong hàng đợi Kafka chưa được xử lý kịp bởi các consumer (Spark/Doris). Lag ở mức 5.244 tin cho thấy hệ thống đang có độ

trễ tiêu thụ tạm thời.

- **Active Doris FE Nodes (1) & BE Nodes (2):** Cụm ODS Doris đang chạy ổn định với 1 Frontend Node (quản lý metadata, phân tích truy vấn) và 2 Backend Nodes (lưu trữ và tính toán).
- **Total Queries (24) & Query Errors (24):** Tổng số 24 truy vấn gửi tới Doris FE và cả 24 đều trả về lỗi (lỗi kết nối hoặc cú pháp trong giai đoạn khởi tạo ban đầu).
- **Kafka Consumer Group Lag by Group/Topic:** Đồ thị trực quan hóa độ trễ tiêu thụ theo thời gian. Đường màu vàng biểu thị tổng lag của `clickstream-events` tăng dần từ 0 lên gần 4.500 tin từ thời điểm 20:37:30 trở đi.
- **Kafka Message Ingestion Rate (per Sec):** Tốc độ nạp tin vào Kafka. Đường màu vàng đại diện cho luồng `clickstream-events` duy trì cực kỳ ổn định ở mức **10 messages/s** (khớp với tốc độ giả lập `PUBLISH_RATE_HZ=10`), luồng `new-users` (màu tím) duy trì ở mức thấp ~0.5 messages/s.
- **Apache Doris FE JVM Memory Usage:** Bộ nhớ JVM của Doris Frontend duy trì ổn định mức **8 GB**.
- **Apache Doris BE Allocated Memory:** Bộ nhớ RAM phân bổ của Doris Backend dao động ổn định trong khoảng **1.05 GB - 1.07 GB** và tăng đột biến lên **1.14 GB** ở cuối chu kỳ (20:39:00) do tải nạp/nén dữ liệu ngầm (Compaction) hoạt động.

5. BENCHMARK VÀ ĐÁNH GIÁ HỆ THỐNG (Đang cải thiện)

Cấu hình thử nghiệm (Docker Limits)

- **Doris FE/BE:** 1 node FE (1 Core, 1.5GB RAM), 2 nodes BE (2 Cores, 2GB RAM mỗi node).
- **Spark/Kafka/Debezium:** Spark Worker (2 Cores, 2GB RAM), Kafka/Debezium (1 Core, 1.5GB RAM).

Tóm tắt kết quả đo lường (Trích xuất từ [demo/result.txt](#))

1. Quy mô dữ liệu & Hiệu năng nạp (Ingestion):

- **Quy mô:** Hệ thống lưu trữ ~930k dòng (lớn nhất là `fact_inventory_items` với 487,901 dòng; `dwd_clickstream_events` đạt 18,048 dòng).
- **Độ trễ Clickstream (Avg / P50 / P99):** **13.12 giây / 3.93 giây / 139.83 giây**.
- **Nạp CDC Routine Load:** Chạy ổn định ở **0% lỗi**, tốc độ nạp cao nhất đạt **202 rows/s** (bảng Inventory).

2. Độ trễ truy vấn đơn (Single Query Latency):

- **Clickstream (CS-1 → CS-4):** Từ **1.5ms** → **4.0ms** (câu CS-5 Session phức tạp tốn **35.6ms**).
- **CDC / OLAP JOIN (CDC-1 → CDC-5):** Phản hồi cực nhanh từ **3.3ms** → **4.7ms**.

3. Chịu tải đồng thời (Concurrency - QPS / P99 Latency):

- **Clickstream:** Đạt đỉnh **683.8 QPS** (Concurrency 3). Ở Concurrency 5 đạt **561.4 QPS** và P99 tăng lên **98.4ms** do nghẽn tài nguyên.
- **CDC / OLAP JOIN:** Scale cực tốt, đạt tối đa **1313.4 QPS** (Concurrency 5) với P99 chỉ **7.2ms**.

4. Tối ưu hóa tài nguyên:

- Tiết kiệm **100% CPU Spark** khi rồi (giảm còn **0.21% CPU**) nhờ chuyển CDC sang Doris Routine Load.
- Cụm Doris FE/BE đạt hiệu suất sử dụng phần cứng tối ưu (**94.52% CPU**).

[!NOTE] *Nguồn gốc số liệu:* Đo lường tự động qua script [demo/benchmark.py](#) (kết nối Doris FE port 9030 qua MySQL protocol, đo 1 warm-up + 5 measured runs, và đo concurrency bắn phá liên tục trong 5 giây) và theo dõi tài nguyên qua [docker stats](#).